

Deep learning software stacks for analogue in-memory computing-based accelerators

Corey Lammie  , Hadjer Benmeziane, William Simon , Elena Ferro , Athanasios Vasilopoulos , Julian Büchel, Manuel Le Gallo , Irem Boybat  & Abu Sebastian  

Abstract

Analogue in-memory computing (AIMC) is an emerging computational paradigm that can efficiently accelerate the key operations in deep learning (DL) inference workloads. Heterogeneous architectures, which integrate both AIMC tiles and digital processing units, have been proposed to enable the end-to-end execution of various deep neural network models. However, developing a software stack for these architectures is challenging, owing to their distinct characteristics — such as the need for extensive or complete weight stationarity and pipelined execution across layers, if maximum performance is to be achieved. Moreover, AIMC tiles are inherently stochastic and hence introduce a combination of stochastic and deterministic noise, which adversely affects accuracy. As a result, existing tools for software stack development are not directly applicable. In this Perspective, we give an overview of the key attributes of DL software stacks and AIMC-based accelerators, outline the challenges associated with designing DL software stacks for AIMC-based accelerators and present opportunities for future research.

Sections

Introduction

Overview of deep learning software stacks

Key attributes of AIMC-based accelerators

Deep learning software stacks for AIMC-based accelerators

Research opportunities

Key points

- Analogue in-memory computing (AIMC)-based accelerators, comprising AIMC tiles and digital processing units (DPUs), can be used to perform end-to-end execution of deep learning (DL) inference workloads, with low latency and high energy efficiency.
- The large-scale adoption of AIMC-based accelerators demands efficient techniques and tools for the co-design of software and hardware, in addition to sophisticated software stacks, which abstract the deployment of diverse deep neural network (DNN) models.
- AIMC-based accelerators based on non-volatile memory have unique attributes, such as fixed weight stationarity and noise arising from analogue computation, and these have a substantial impact on how software stacks are developed for them.
- In most work on memory devices for AIMC or the design of AIMC-based accelerators, the complexity of DL software stack development and, consequently, the amount of time required are given insufficient consideration.

Introduction

In recent years, driven by increases in computational capability and by architectural innovation, modern deep learning (DL) inference workloads have become widespread. Although general-purpose processors, namely central processing units (CPUs), can be used to perform DL inference, they are relatively energy-inefficient and suffer from performance issues. More specialized programmable parallel processors – for example, graphics processing units (GPUs) – are much more performant for highly parallelizable workloads; however, achieving optimal performance with them comes at the cost of high power consumption and reduced energy efficiency¹. To achieve maximum performance and energy efficiency requires customized hardware accelerators, carefully designed to optimally execute end-to-end DL inference workloads².

An emergent class of customized hardware accelerators relies on the paradigm of in-memory computing (IMC), which addresses the processor–memory dichotomy in modern computing systems. Certain computational operations, such as matrix–vector multiplication (MVM), are performed in place in the memory itself by exploiting the physical attributes of the memory devices³. Current state-of-the-art GPUs achieve >5 tera operations per second per watt (TOPS W⁻¹), for example ≈8.35 TOPS W⁻¹ for an **NVIDIA H100 NVL at 400 W**, and edge tensor processing units⁴ achieve 2 TOPS W⁻¹ at 2 W. In contrast, recent IMC prototype chips have been shown to achieve a sustained >10 TOPS W⁻¹ (refs. 5,6) for certain DL workloads, and heterogeneous architectures are projected to achieve >50 TOPS W⁻¹ (ref. 7), highlighting their strong potential for addressing the increasing energy overheads of modern DL workloads¹.

IMC architectures can be broadly categorized as digital IMC and analogue in-memory computing (AIMC). For digital IMC, the multiply–accumulate (MAC) operation associated with MVM operations is implemented using digital logic gate-based multiplier and adder circuitry; for AIMC, analogue currents are accumulated during operation. Digital IMC and AIMC based on static random-access memory (SRAM) are mature enough for near-term commercialization.

However, a notable limitation is that SRAM-based arrays lack the density to store all the weights of most deep neural networks (DNNs) on a single die within practical chip area constraints. Hence, weight reloading is typically performed as a result of limited weight capacity. A potential solution is to transition to non-volatile memory technologies, such as resistive memories (including resistive random-access memory and phase-change memory) or flash memory, as they offer much higher memory density than SRAM. The central component of such an AIMC-based accelerator is the AIMC tile. AIMC tiles arrange compute units in a crossbar topology. In one implementation, the conductance of each device can be modulated to represent specific weight values⁸. During MVM operations, inputs are applied as voltages through digital-to-analogue conversion, while the resulting currents accumulate. These currents are converted to digital outputs using analogue-to-digital conversion. AIMC enables constant-time computation that is invaluable in applications requiring high throughput, such as DL inference.

Although AIMC tiles enable the highly efficient realization of MVM operations to perform end-to-end execution, there is a need to support additional operations associated with DL inference^{9–12}. This motivates the use of a heterogeneous architecture (Fig. 1a) that complements AIMC tiles with various digital processing units (DPUs). Depending on the desired performance and flexibility, these can be general-purpose processors or highly customized digital compute units¹³. It is also critical to seamlessly integrate these processing units (AIMC tiles and DPUs) with a high-bandwidth digital communication fabric⁹.

Figure 1b shows the typical life cycle of a DNN model: among the critical components are DL software stacks. From the point of view of a typical end user performing offline inference or serving a model, DL software stacks automate and abstract the steps required to deploy diverse DNN models to specific hardware back ends, which execute the model. For customized hardware accelerators, this is problematic, because specialized firmware and software must be developed for integration with existing DL software infrastructure – and the complexity of this task and the effort required are often overlooked. Consequently, in this Perspective we focus on the challenges and future research directions related to DL software stacks for AIMC-based accelerators adopting a weight stationary data flow. We delineate the composition of these software stacks, offering insights into their unique design considerations. Critically, we emphasize that, although AIMC-optimized software stacks share foundational principles with stacks designed for other custom hardware accelerators, the distinct operational and architectural requirements of AIMC necessitate nuanced adaptations that pose challenges for infrastructure reuse, and underscore the need for customized development and a robust and engaged open source community.

Overview of deep learning software stacks

In this short overview, we answer two critical questions: what is the objective for DL software stacks, and what are their inputs and outputs? We describe the composition of DL software stacks, introduce the concept of intermediate representations (IRs) and highlight the importance of feedback-directed optimization (FDO). We also discuss how pre-existing frameworks and tools can be used to accelerate DL software stack development.

Basic concepts

The objective for a DL software stack is to automate and abstract the deployment of diverse DNN models – ideally, unbeknownst to the

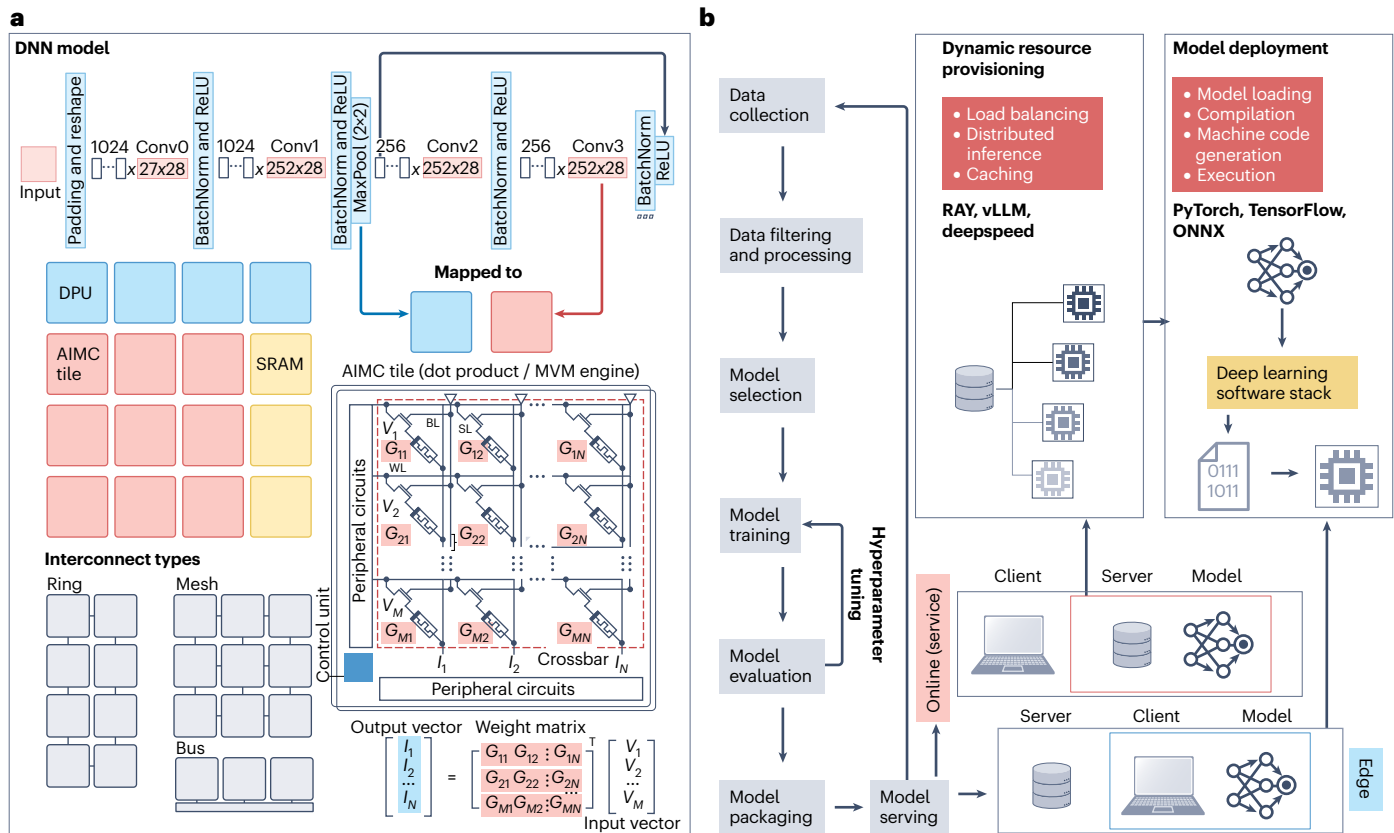


Fig. 1 | Deployment of a DNN model to an AIMC-based accelerator and the model's complete life cycle. a, A deep neural network (DNN) model can be deployed to an analogue in-memory computing (AIMC)-based accelerator that comprises AIMC tiles, various digital processing units (DPUs) and static random-access memory (SRAM) banks. The underlying compute mechanism of in-memory computing (IMC) is shown, and also different types of interconnects that facilitate the movement of data between processing units and memory blocks. MaxPool refers to a pooling operation that calculates the maximum value for patches of a feature map, and uses this to create a down-sampled feature map. For an individual AIMC tile, bit lines (BL), select lines (SL), word lines (WL), voltages (V), device conductances (G) and accumulated currents (I) are labelled. **b**, In the typical life cycle of a DNN model, data are first collected, then filtered

and processed. Next, model selection is performed. The selected model is trained and evaluated, then packaged. Models can be served in both edge and online scenarios; in an online service-based scenario, dynamic resource provisioning is required, which entails load balancing, distributed inference and caching steps. In both edge and online scenarios, model deployment to a desired hardware back end is performed. This requires a sophisticated deep learning (DL) software stack, which consumes a DNN model and generates machine code. BatchNorm, batch normalization layers; ConvN, Nth convolutional layer; MVM, matrix–vector multiplication; ONNX, Open Neural Network Exchange; RAY, software library (<https://github.com/ray-project/ray>); ReLU, rectified linear unit activation function; vLLM, software library (virtual large language model).

end user. The primary aim is to maximize accuracy and performance, subject to hardware constraints. A DL software stack takes as input a DNN model, comprising a model architecture, associated parameters and a forward propagation method which governs the behaviour of the forward passes. DNN models are typically defined using high-level tensor-based frameworks that provide the flexibility to represent models using a combination of tensor-based and layer-based operations. For example, using one of these frameworks, PyTorch¹⁴, a fully connected (or linear) layer with an arbitrary number of input and output features can be implemented functionally using `torch.nn.functional.Linear(input, weight, bias)` or by using tensor primitives: `torch.matmul(input, weight.T) + bias`.

These operations, and their relation to each other, can be decomposed at different levels of abstraction using common sets of operations for compilation and execution. In addition to the model definition, a system definition, in a data-interchange format (for example, JSON),

must also be provided. (In this context, a system refers to the comprehensive set of components, processes and architecture that collectively enable the execution of DNN models.) Depending on the targeted accelerator, the system definition should contain varying degrees of information about the system components and interconnects. As an example, for a model to be deployed where weights and activations of different layers are stored using different precisions during operation, the definition could contain information such as quantization bounds, and the scales and offsets required to convert between representation domains. The output of a DL software stack is machine code that configures the hardware and executes the DNN model. The configuration can occur initially, periodically, conditionally or during operation, depending on the mode of operation.

Before detailing the composition of a DL software stack (Fig. 2a), it is important to first explain what an IR is and define some related terminology. An IR is broadly defined as a specific representation,

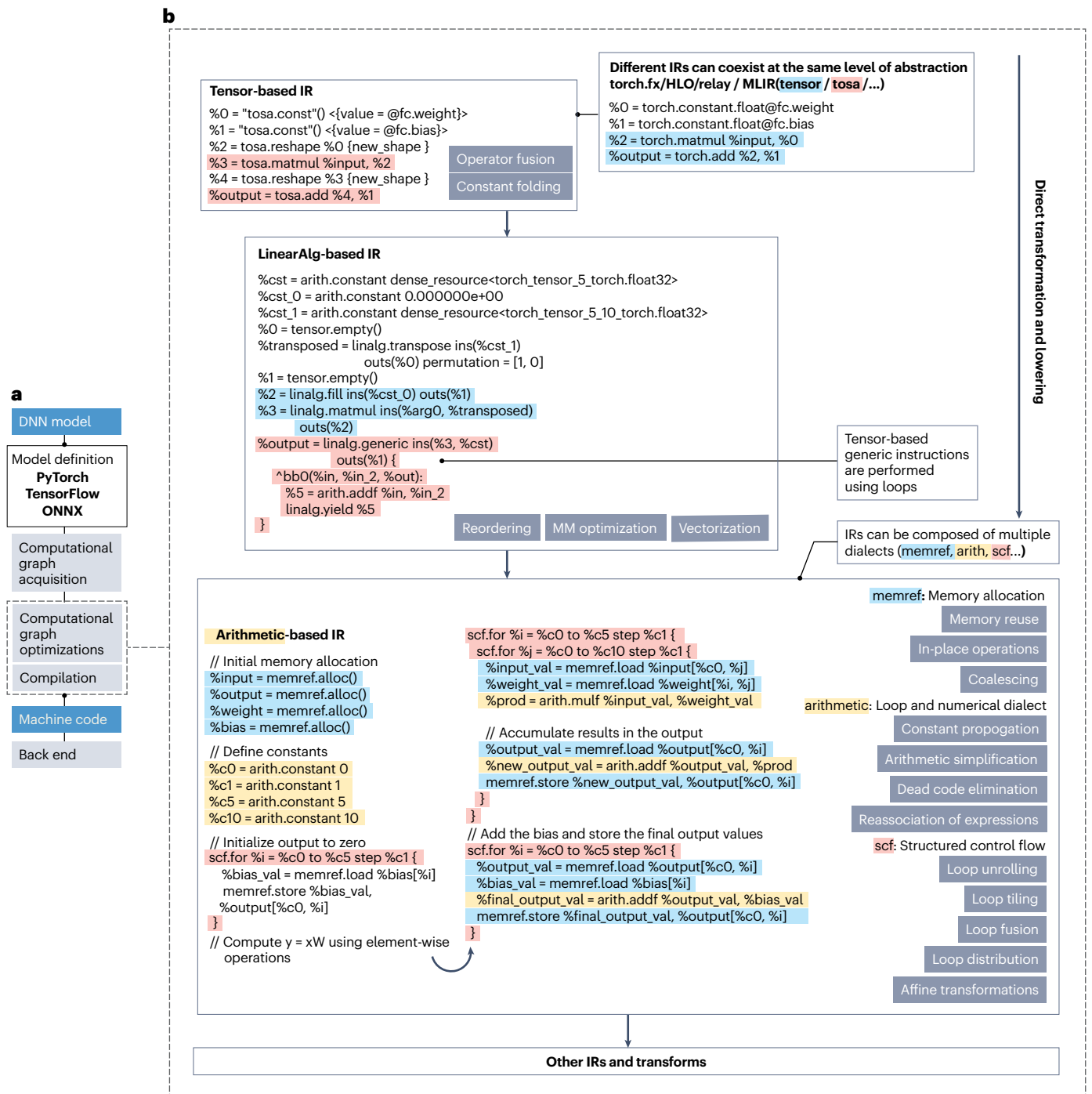


Fig. 2 | High-level composition of a standard deep learning software stack.

a, At a high level of abstraction, a traditional deep learning (DL) software stack, which consumes a deep neural network (DNN) model, comprises blocks for computational graph acquisition and optimization, and compilation. The back end consumes machine code and realizes model execution. **b**, DNN models can be expressed with different intermediate representations (IRs) using dialects with varying degrees of abstraction. At each level of abstraction, different transformations and optimizations can be applied. As IRs are lowered, more hardware-specific information is introduced and optimizations become more granular in nature – for example, tensor-based IRs can be used to perform

operator fusion and constant folding, whereas arithmetic-based IRs can be used to perform efficient memory allocation and loop unrolling. Different IRs can coexist at the same level of abstraction (two different tensor-based IRs are shown), and IRs can be composed of multiple dialects. For example, the arithmetic-based IR includes operations from the memref, scf and arithmetic dialects. Many different passes, in addition to the one shown, can be used to perform compilation. (In the Linalg MLIR (Multi-Level Intermediate Representation) dialect, %bbx refers to the xth basic block or basic block ID and %ins and %outs refer to inputs and outputs of a computation or operation.) ONNX, Open Neural Network Exchange.

structure or encoding of data, used in the design and implementation of compilers, that serves as an intermediary between the input and the generated machine code¹⁵. IRs can take various forms, such as parse trees or abstract syntax trees, but in the context of DL model compilation we refer to an IR as an arbitrary sequence of related operations, which can be represented using a directed graph. To clearly distinguish between high-level IRs used to define models in tensor-based frameworks and those used for compilation, we will refer to the former as computational graphs and the latter as IRs.

Each IR enables the definition of new operations, as well as attributes and types¹⁶, which can be used, for example, to specify the iteration space or the types and access patterns of the inputs and outputs for a specific operation. IRs can be defined at different levels of abstraction, with varying degrees of hardware specificity. As an example, at one level of abstraction, a fully connected layer can be represented using transpose, matrix multiplication and broadcast element-wise addition operations. At another, loop unrolling can be performed, and the matrix multiplication operation can be decomposed into a sequence of dot-product operations with hardware specificity, such that the utilization of the underlying hardware is maximized by the loops.

‘Dialects’ are used to group different IR objects into a common declarative region. Within these regions, commonly referred to as namespaces, transformations and operations that can be performed on the IR, valid IR operator types, attributes, traits, interfaces and passes – that is, sequences of transformations and operations – can be defined as objects. Hence, a dialect can be considered a superset of an IR in the sense that it extends an IR with additional constructs while adhering to the IR’s infrastructure and principles. Although these definitions belong to exactly one dialect, definitions from different dialects can coexist at any level of abstraction of an IR at any time. The decomposition from one dialect to another at a lower level of abstraction is referred to as ‘lowering’; multiple lowering steps involving one or more dialects are referred to as a ‘pass’. Figure 2b shows the lowering of a simple DNN model comprising a single linear layer, represented using two tensor-based dialects, to a linear algebra-based dialect and an arithmetic-based dialect. These can be further lowered and interfaced with tools such as LLVM, to be used to perform machine code generation. (LLVM and other related tools will be discussed in greater detail below.)

At a high level, DL software stacks comprise a front end, to which inputs are fed for processing, and a compiler, which generates machine code. Although these terms are inconsistently used in the literature, the front end usually refers to the high-level application programming interface or an interface where users provide a DNN model, data-processing pipelines and optional retraining logic (for example, quantization-aware training). We adopt this definition. We consider the compiler as being responsible for transforming and optimizing the model from a high-level IR to optimized, low-level machine code. Compilation can be performed statically, prior to execution (‘ahead of time’) or during execution (‘just in time’). Hybrid approaches are also becoming popular: these support dynamic and auto-regressive computation by statically compiling and generating machine code for different network components and scenarios, and are stitched together using just-in-time compilation logic. This kind of flexible approach is inspired by recent work in other domains that combines both ahead-of-time and just-in-time compilation approaches¹⁷.

Finally, there is the back end. In the literature, at times, the back end is stated to perform computational graph optimizations¹⁵, which use symbolic reasoning to improve computation efficiency, model

transformations and overall execution performance. Additionally, it is stated to perform a number of passes on the model IR. These optimizations and passes progressively introduce hardware constraints and optimize the DNN model for deployment on the underlying hardware – eventually generating executable machine code which contains sequences and loops of instructions and routines in a directly interpretable form by the underlying hardware. We consider the back end as being solely responsible for providing the execution environment for the machine code, enabling it to interact directly with hardware resources. In other words, the back end ensures that the machine code knows how to run on the specific hardware it is assigned to, and provides the tools to execute the code efficiently and effectively.

Feedback-directed optimization

The ideal scenario for deploying a model is a one-shot process, in which all necessary steps – such as hardware mapping, schedule optimization and machine code generation – are performed only once. However, in reality, strong interdependencies between deployment stages make this difficult. Decisions such as operator mapping and placement onto processing units affect scheduling, memory access and machine code structure, leading to non-linear and unpredictable performance outcomes.

Heuristic-based optimization alone is often insufficient¹⁸, as it cannot fully capture the complexity of these interactions. Instead, a dynamic feedback mechanism is essential for refining decisions iteratively. Full-scale evaluation after every design choice is impractical for large-scale DL inference, especially with just-in-time compilation. Therefore, FDO is used to explore design choices – such as operator placement and precision settings – while balancing trade-offs such as accuracy, power consumption and throughput¹⁹. FDO is performed by introducing optional feedback loops between different compilation steps; the loops typically use proxy models or estimation tools to quickly approximate the impact of the decisions made at different steps.

Existing frameworks and tools

There exist numerous frameworks and tools, which can be extended to support different hardware back ends. For all these frameworks, computational graphs and IRs are critical to their operation, acting as a bridge between DNN models and machine code. We categorize existing frameworks and tools into two distinct categories: first, frameworks that can be used to define DNN models, and the tools that can convert between them; and second, frameworks and tools used for model compilation. We also discuss MLIR (Multi-Level Intermediate Representation), an innovative software infrastructure specifically designed to support compiler development.

The most popular frameworks for defining DNN models are PyTorch¹⁴, TensorFlow²⁰ and JAX. Although these frameworks share similar layer and function primitives and operate at comparable levels of abstraction, converting DNN models between them remains a complex challenge. This difficulty arises from key differences in their underlying architectures, execution paradigms and application programming interface designs. For example, PyTorch’s eager execution model contrasts with TensorFlow’s historic reliance on static graphs, and each framework also has distinct layer definitions, default settings and hardware-specific optimizations. These differences make direct conversion between models nuanced and have driven the development of standardized, universal IRs^{21,22}, with the [Open Neural Network Exchange](#) (ONNX) format being the most prominent.

These universal IRs aim to act as intermediaries that enable interoperability across the computational graphs of different frameworks, simplifying model conversion by translating high-level framework-specific constructs into standardized representations.

Among the most popular frameworks and tools that can be used to perform compilation, Apache TVM is an open source DL compiler stack that relies on an IR called ‘Relay’²³. TVM performs a series of transformations on this IR, such as algebraic simplifications, loop unrolling and memory optimizations, and finally lowers the IR into machine code. Through this pipeline, TVM supports a wide range of hardware back ends, including CPUs, GPUs and customized hardware accelerators. The LLVM Project is a collection of modular and reusable compiler and tool-chain technologies²⁴, and is commonly used by other frameworks, including TVM, for low-level IR optimizations and machine code generation. Tensorflow XLA (Accelerated Linear Algebra) is Google’s framework for optimizing and compiling DL models, particularly those created in TensorFlow, and uses an IR tailored for Google’s tensor processing unit and other hardware accelerators. For CPU and GPU back ends, XLA uses LLVM for low-level optimization and machine code generation. Glow (Graph Lowering for Optimized Wireless) is a compiler framework designed by Meta²⁵. It serves as a machine learning-oriented layer on top of LLVM, leveraging LLVM’s mature compiler technology to achieve high performance and flexibility. Glow uses a two-step compilation approach: first, it compiles neural network models from high-level frameworks into a hardware-agnostic IR specific to Glow that it can optimize. Once Glow has optimized the IR, it then leverages LLVM’s back end for the final lowering process, translating the optimized IR into machine code for specific hardware back ends. NVIDIA TensorRT is a high-performance DL inference optimizer and run-time library developed by NVIDIA, specifically tailored for NVIDIA GPUs²⁶. OpenVINO is a toolkit developed by Intel that accelerates DL inference across a wide range of Intel hardware²⁷. It is designed with a particular focus on applications involving computer vision and edge computing and supports various Intel platforms, including CPUs, integrated GPUs, vision processing units and field programmable gate arrays. One of OpenVINO’s strengths lies in its model optimizer, which pre-processes and streamlines the model before it reaches the inference engine, applying optimizations that reduce computation and memory overhead. The optimized model is then run on the OpenVINO Inference Engine, a run time that dynamically selects the best execution strategies for the available Intel hardware.

All of the aforementioned compilation frameworks and tools operate using one or more central IR, containing a complete set of instructions which can be modified and extended. Motivated by the lack of standardization between the different stacks, and the need for seamless translation to machine code, MLIR¹⁶ was developed to offer a standardization of the compiler infrastructure by providing a unified interface to define dialects. These dialects help represent computations across multiple levels of abstraction – from high-level DL operations to low-level machine code generation. This allows developers to optimize computations at the most appropriate level, whether refining algorithmic operations for domain-specific needs or tailoring machine code to leverage the unique features of customized hardware architectures. By unifying these abstraction levels within a single framework, MLIR not only streamlines the compilation process but also enhances efficiency and performance in ways that traditional, rigid compiler designs cannot. This disruptive infrastructure has fundamentally changed how many recent compilers have been designed and developed (for example, XLA), and is becoming increasingly widely adopted.

Key attributes of AIMC-based accelerators

The following key characteristics of AIMC-based accelerators add complexity to the development of DL software stacks.

Co-location of heterogeneous processing units

In AIMC-based accelerators, DPUs play a crucial part in handling operations that AIMC tiles cannot execute, such as activation functions and attention computations for transformer-based models. Additionally, DPUs can extend the effective model size by managing portions of the model that exceed the weight capacity of AIMC-based accelerators. Another important use of DPUs is accelerating small MVM kernels when execution on AIMC would be slower, or supporting MVMs for which analogue noise sensitivity is a concern²⁸. There are a broad range of DPU types, from general-purpose core-based units that accelerate a wide range of operations to more specialized units, for certain specific operations, that are optimized for lower latency^{9,13,29}.

Noise

AIMC introduces both deterministic and stochastic noise, which affects its accuracy. Factors such as memory device stochasticity, temporal and thermal conductance drift, inaccuracies in analogue-to-digital and digital-to-analogue conversion, and IR drop can introduce error in AIMC tiles. Deterministic noise sources, such as quantization noise, can be modelled exactly a priori – meaning that prior to deployment, their impact on accuracy can be evaluated and can be guaranteed during operation. By contrast, only the average and variance behaviour of stochastic noise sources can be modelled³⁰, and although the effect of these on accuracy can be mitigated using different techniques (such as hardware-aware (HWA) training³¹, post-placement calibration methods and chip-in-the-loop retraining; Box 1), AIMC tiles remain unsuitable for particularly sensitive operations. AIMC-based architectures address this problem by offloading critical operations – those required to be executed using high-precision computations – to DPUs. In this configuration, AIMC tiles are utilized for less sensitive static MVMs, whereas more sensitive static MVMs, along with dynamic MVMs, accumulations, activations and normalization operations, are executed on DPUs.

Fixed weight stationarity

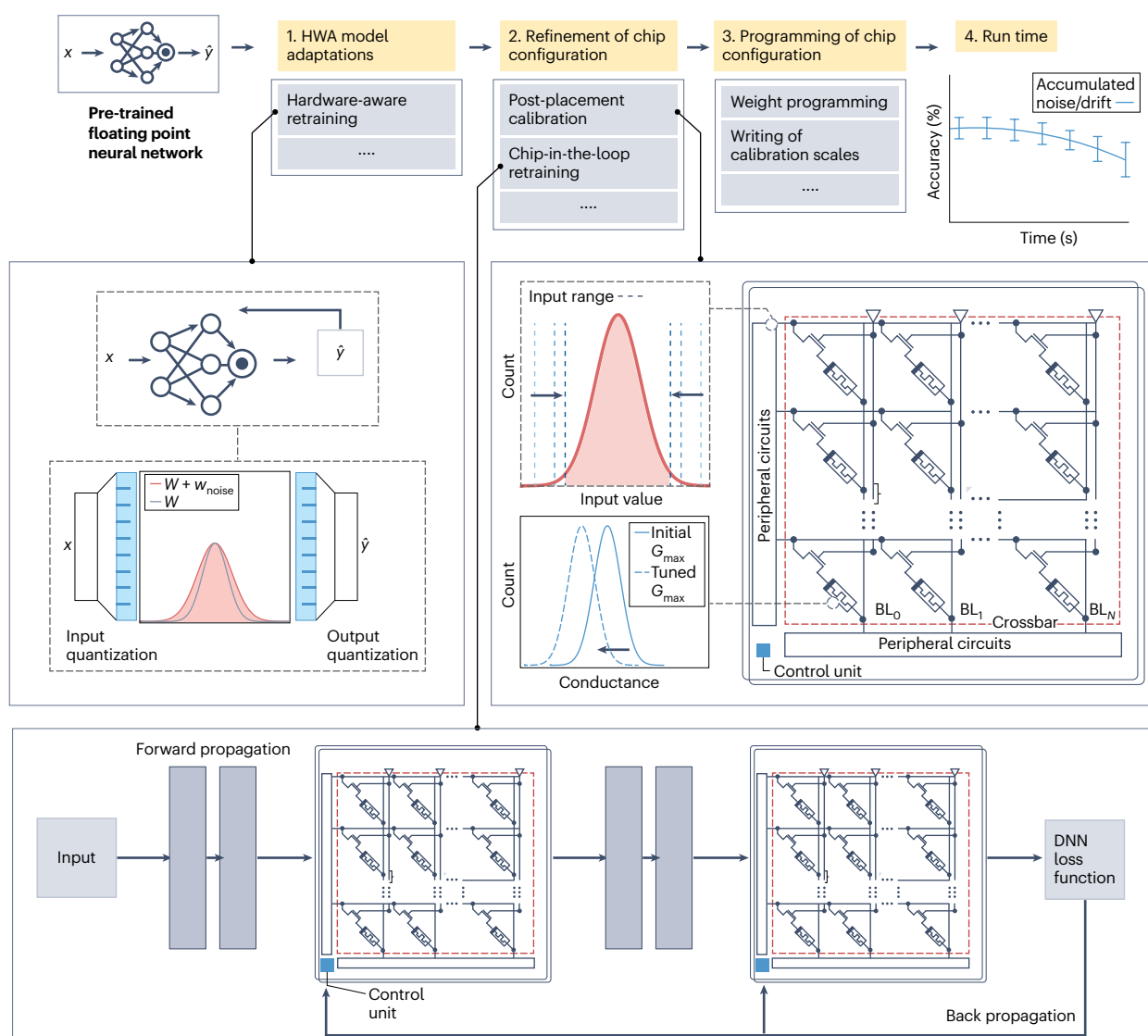
Owing to the relatively prolonged time required to program non-volatile memory devices and their limited endurance³², compared with other technologies such as SRAM, weight reloading cannot be readily performed during operation. Hence, for most AIMC-based accelerators, a substantial number (if not the totality) of weights are first transferred and programmed to memory devices, and then computation is performed. However, DPUs can be leveraged to extend the effective model size by handling parts of the model that exceed the weight capacity of all the AIMC tiles. Reprogramming AIMC tiles is time and energy-inefficient, but DPUs can efficiently fetch weights from memory, meaning that they can be used to reload weights, at the cost of reduced performance and efficiency.

To achieve optimal performance when weights are stationary, pipelined execution across layers is required (inter-module parallelism, where each layer or group of layers is considered a module). For instance, during the first pipeline step, for the first batch, the first layer is executed; for the second pipeline step, for the first batch, the second layer is executed, and for the second batch, the first layer is executed; and so on. Additional scheduling modifications are required to effectively pipeline execution across layers: most existing frameworks

Box 1 | Commonly adopted hardware-aware model adaptation and chip configuration refinement methods

Before pre-trained floating-point neural network weights are deployed to an analogue in-memory computing (AIMC)-based accelerator, they can be retrained in a hardware-aware (HWA) manner (see the figure). During this process, mappable weights (W) are converted to representative conductance values, Gaussian noise (w_{noise}) is injected into them during forward passes and a number of non-idealities are simulated, for example, inputs (x) and outputs (y) are quantized. These weights are updated in floating-point precision using the noisy outputs ($\hat{y}$). w_{noise} is sampled from a Gaussian distribution during inference. It is sampled either per batch or per tensor.

Post-placement calibration methods can be used to recover deep neural network (DNN) accuracy losses introduced by non-deterministic noise sources. Two common methods are input range and conductance range calibration. The former tunes the input range of each tile, navigating the trade-off between information loss and precision. The latter tunes the maximum conductance range (G_{max}) of the N th bit line (BL_N) to avoid excessive analogue-to-digital converter current saturation. Alternatively, chip-in-the-loop retraining can be performed, which performs on-chip matrix-vector multiplication (MVM) during forward propagations and utilizes the DNN loss function output.



assume weight reloading is performed, and hence are designed to parallelize operations within layers, not across them.

Analogue-to-digital domain conversion

To implement an MVM operation using an AIMC tile, numerous steps must be performed. First, for each row of the input matrix, values must be transformed (scaled, clipped and quantized) to an appropriate range. Typically, a pulse-width modulation or binary voltage driver is used to send voltage pulses to each word line (row) of the AIMC tile and currents are accumulated across each bit line (BL) (column). Based on the active weight region, different word lines and BLs must be activated. These analogue currents are then converted back to digital values using analogue-to-digital conversion. Affine scales and offsets are applied to compensate for non-idealities and the resulting values are transformed to another appropriate range and numerical format for further processing³³. The final transformation scale, which typically can be merged with the affine scale, is a product of the input, weight and conductance ranges. All of these steps can be orchestrated using a reconfigurable local controller to avoid excessive communication overhead, but this introduces a control hierarchy, which increases the complexity of generating machine code.

Inter-tile and intra-tile weight mapping

Mapping weights to AIMC tiles is non-trivial, because both the tile location in relation to DPUs as well the physical location of mapped weights within a tile affect performance. For the former consideration, the weights of larger layers can be distributed across multiple tiles, but this introduces additional communication and aggregation penalties – particularly if the tiles, or the DPUs used to perform aggregation, are not located in close proximity to each other. In the latter situation, within a tile, the inference of different weight regions can be parallelized if they have non-overlapping row and column regions. If two weight regions share the same input, their inference can also be parallelized, by mapping them adjacently. If a tile is under-utilized, then the performance of the accelerator is negatively impacted. Conversely, if many weight regions from different layers are mapped to the same tile, pipelining across input batches becomes increasingly difficult. Finally, weight duplication can be performed to improve throughput, at the cost of increased area and power.

Calibration scales

In addition to the aforementioned affine scales and offsets, there are numerous other calibration scales, which must be determined before DL inference can be performed. Some of these are specific to each hardware instance (such as the gain of each analogue-to-digital converter) and can be calibrated using synthetic data, then fixed for different network deployments¹². Others are network-specific (for example, the input range of each AIMC tile) and must be determined a priori or during compilation, after the layer weights are mapped.

Deep learning software stacks for AIMC-based accelerators

The key blocks that make up DL software stacks for AIMC-based accelerators (Fig. 3) are not dissimilar to those for customized heterogeneous accelerators. The extent of the modifications needed to adapt existing infrastructure, for the components within each block in Fig. 3 to be reusable with AIMC-based accelerators, varies considerably. Although most blocks are shared with a standard DL software stack, some require substantial modifications.

Before computational graph acquisition and optimizations can be performed, numerous HWA model adaptations are required to account for hardware-specific characteristics and training adaptations. As an example, additional HWA training (Box 1) is often required for pre-trained floating-point networks owing to deterministic and stochastic noise sources. For some hardware architectures, at the same time, calibration scales (such as those required for affine correction) can be learned using back propagation. Modification of network weights and pruning can be performed at different stages with varying degrees of supervision, for example, during training with labelled data (quantization-aware training) or post training (post-training quantization). In either case, the behaviour of these steps is governed by information in the provided system definition. The co-location of heterogeneous processing units necessitates heterogeneous mapping of layers to AIMC tiles and DPUs. This can be performed using simple heuristic-based²⁸ or more sophisticated approaches³⁴.

As discussed earlier, computational graph optimizations are a core ingredient of any DL software stack. After computational graph acquisition, formal mathematical techniques, including algebraic manipulation, are leveraged to perform optimization. These transformations are then applied to improve performance metrics, conserve computational resources and streamline model calculations. Key transformations include expression simplification, dimension inference and shape optimization, and operator fusion – which entail the reduction of complex expressions to simpler forms, avoidance of redundant reshaping operations and combination of multiple operations into compound operations.

Traditional IRs need customization to support hardware-specific features of heterogeneous accelerators, such as extensive or complete (fixed) weight stationarity, and inter-tile and intra-tile weight mapping. High-level IR optimizations are then performed to implement hardware-specific templates to perform operator fusion, transformations and graph reconstructions. These transformations are also used to lower a layer-based IR to an operation-based IR, on which low-level IR optimizations are performed. These optimizations include placement, post-placement calibration, memory management and scheduling steps. Mapping entails the association of one or more operations to a type of processing unit, whereas placement entails the assignment of a mapped operation to a processing unit at a specific physical location. Chip configuration refinement steps are performed to improve accuracy by tuning some scales and parameters using prior hardware information or feedback from the hardware directly. Memory management and scheduling determine the sequence of operations during execution and how different memory hierarchies, such as caches, are utilized.

Finally, an optimized, operation-based IR is converted into machine code tailored for the AIMC-based accelerator. This machine code encompasses control sequences and commands for both primary operations and low-level routines, for example, analogue-to-digital domain conversion. The increased diversity of AIMC-based hardware architectures can introduce challenges during this step, as traditional compiler back ends assume a general-purpose instruction set architecture (ISA). For AIMC-based hardware architectures with finite state machine (FSM)-driven control logic, specialized compilation techniques, such as microcode generation or hardware description languages, may be required. More complex architectures with coexisting FSM-driven control logic and ISA-based execution models require FSM-controlled operations to be aligned with ISA-driven execution, which needs careful scheduling and control flow management.

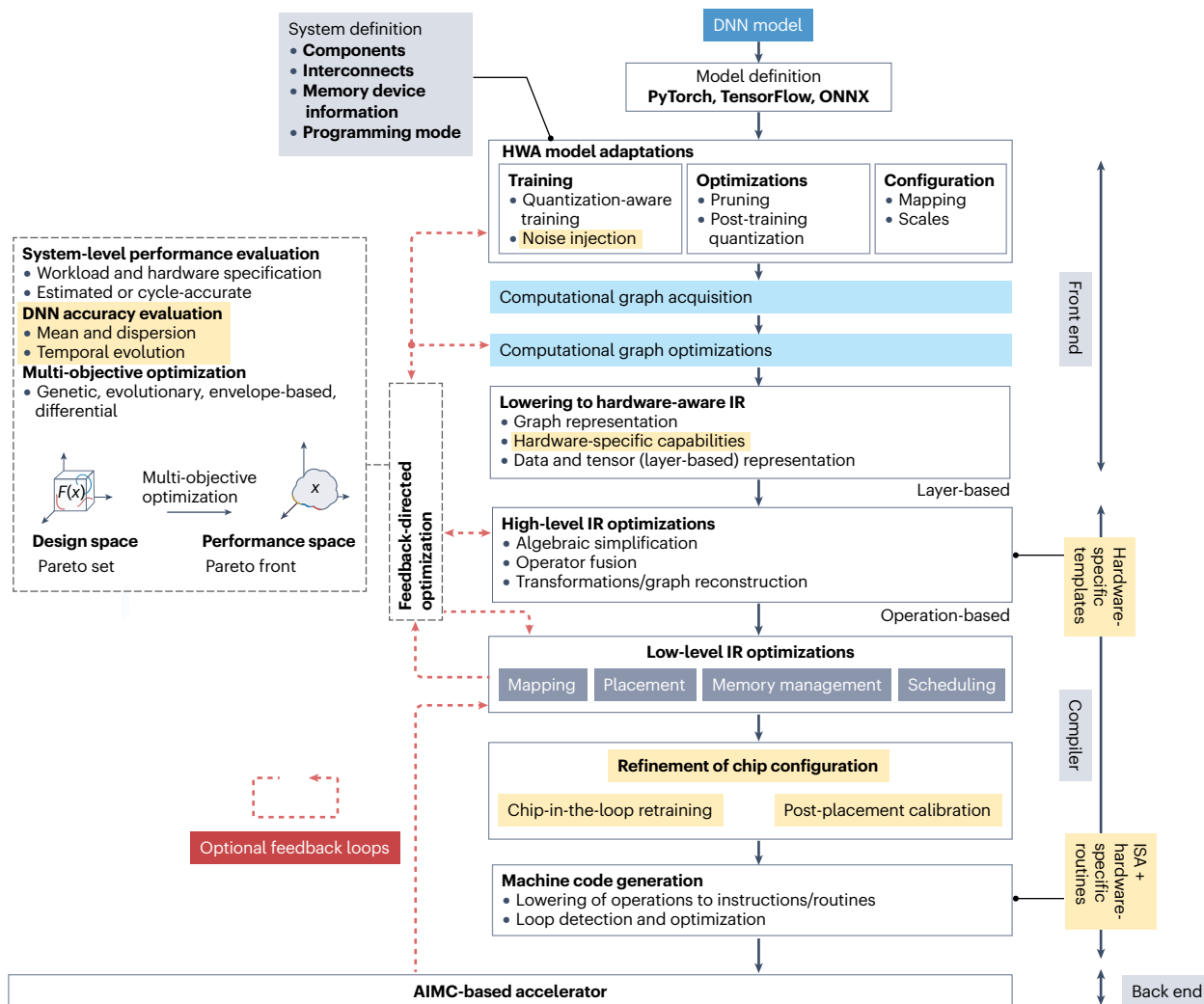


Fig. 3 | Composition of a deep learning software stack and back end for an AIMC-based accelerator. Components within the blocks that require extensive modification to accommodate analogue in-memory computing (AIMC) are highlighted (yellow). To perform end-to-end model deployment, high-level deep neural network (DNN) model and system definitions – containing information about system components, interconnects, memory device information and the desired programming mode – are provided to the front end. The compiler

consumes hardware-specific graph optimization templates and an instruction set architecture (ISA), and generates machine code. Dashed lines indicate steps that are optional. Feedback loops can be introduced between different blocks, and used to perform DNN accuracy and system-level performance evaluations – usually through estimations. Multi-objective optimization is performed using feedback-directed optimization (FDO). HWA, hardware-aware; IR, intermediate representation; ONNX, Open Neural Network Exchange.

Additionally, efficient coordination between FSMs and ISAs requires the careful handling of control signals, memory access patterns and timing constraints, which can complicate code generation and execution.

Feedback-directed optimization for AIMC-based accelerators

FDO for AIMC-based accelerators relies on multi-objective optimization targeting both DNN accuracy and system-level performance. The degree to which AIMC noise affects DNN accuracy may vary depending on how the various computational operations are mapped and placed onto the heterogeneous processing units. For instance, operations that are more sensitive to noise may need to be offloaded to DPUs, whereas less sensitive operations can remain on AIMC tiles. As mapping

and placement also affect system-level performance, a careful balancing of operations between analogue and digital units is necessary – a task well suited to optimization through FDO, the result being a set of configurations that represent the best possible trade-offs. A hybrid approach, combining static and dynamic optimization, can be used: static compilation pre-optimizes parts of the model, and dynamic adjustments are made during inference, to handle run-time variability.

Being able to precisely estimate DNN accuracy and system-level performance metrics is essential for the effectiveness of FDO in heterogeneous accelerators. To this end, a combination of analytical models, proxy models and simulation-based techniques can be used. For the heterogeneous mapping of weights to AIMC tiles and DPUs, one particularly effective proxy for AIMC energy efficiency is the analogue MAC

Table 1 | Comparison of different kernel acceleration compilation tools and deep learning inference compilation tools for in-memory computing-based hardware

Title or acronym	Year	PyTorch or TensorFlow interface	Inter-module parallelism	Intra-module parallelism	Infrastructure reuse	Weight stationary	Kernels or networks	Open source
Kernel acceleration								
DS Language and Compiler ⁴⁵	2017	✗	✗	✓	–	Full	Primitive mathematics operations	✗
IM Execution of Compute Kernels ⁴⁶	2017	✗	✗	✓	LLVM	Full	Primitive mathematics operations	✗
IMDPP ⁴⁷	2018	✓	✓	✓	TensorFlow DFGs	Full	Primitive mathematics operations, clustering algorithms	✗
TDO-CIM ⁴⁸	2020	✗	✗	✓	LLVM	Full	PolyBench/C code	✗
OCC ⁴⁹	2021	✗	✗	✓	MLIR (Teckyl)	Full	BLAS routines	✓
Deep learning (DL) inference acceleration								
HSCD ⁵⁰	2018	✓	✓	✓	ONNX	Full	MLPs, CNNs	✗
DNN+NeuroSIM ⁵¹ 1.4	2019	✓	✗	✓		Full/partial	MLPs, CNNs, LSTMs	✓
PUMA ⁵²	2019	✓	✓	✓	ONNX	Full	MLPs, CNNs, LSTMs	✓
TC-CIM ⁵³	2020	✓	✗	✓	Facebook TCs	Full	MLPs	✓
XLA-NDP ⁵⁴	2023	✓	✗	✓	Tensorflow XLA	Full	MLPs, CNNs, LSTMs	✗
ACTF-ReRAM ⁵⁵	2023	✗	✗	✓	LLVM	Full	BLAS routines	✗
HTVM ⁵⁶	2023	✓	✗	✓	TVM, DORY	Full/partial	MLPs, CNNs, LSTMs	✓
PIMCOMP ⁵⁷	2023	✗	✓	✓	ONNX	Full	CNNs	✓
ACF-SRAM-CIM ⁵⁸	2024	✗	✗	✓		Full	MLPs, CNNs	✗
CIM-MLC ⁵⁹	2024	✓	✗	✓	ONNX	Full	CNNs, transformers	✗

BLAS, basic linear algebra subprograms; CNN, convolutional neural network; DFG, data-flow graph; DNN, deep neural network; LSTM, long short-term memory; MLIR, Multi-Level Intermediate Representation; MLP, multilayer perceptron; ONNX, Open Neural Network Exchange; SRAM, static random-access memory; TC, tensor comprehension; XLA, Accelerated Linear Algebra.

ratio, which measures the proportion of MAC operations executed on AIMC tiles relative to the total MAC operations required by the model²⁸. This metric provides a rapid assessment of the utilization efficiency of analogue resources and offers valuable insights into potential energy savings and throughput improvements. For example, a higher analogue MAC ratio indicates that a larger fraction of the workload is offloaded to AIMC tiles, potentially reducing power consumption considerably.

To balance model complexity and computational feasibility, a hybrid proxy modelling approach can be adopted, combining lightweight analytical estimations with selective simulation-based validation. This strategy mitigates the risks of oversimplification by employing efficient proxies for routine assessments, whereas detailed simulations validate predictions for critical decisions. For instance, regression-based models predict latency and energy consumption on the basis of the input size, precision settings and layer characteristics, enabling rapid yet reasonably accurate estimations. Two main components should be considered: model features and hardware attributes. Model features include layer dimensions, sparsity levels and precision settings – these directly influence computational cost and memory access patterns. Hardware attributes include the number of AIMC tiles, noise sensitivity, conductance variability and analogue-to-digital conversion precision, as well as DPU capabilities such as computing precision and memory bandwidth.

Surrogate modelling techniques, such as Gaussian process regression and Bayesian optimization, help balance accuracy and computational cost by efficiently exploring the design space on the basis of prior performance observations. Multi-fidelity models dynamically

switch between low-fidelity estimates and high-fidelity estimates on the basis of optimization stage and decision criticality. By integrating these estimation techniques directly into the iterative feedback loops of FDO, the software stack can dynamically adjust mapping, placement and precision settings, ensuring a balanced trade-off between accuracy, power efficiency and throughput. This hierarchical proxy modelling approach ensures computational feasibility while enhancing the noise robustness of large-scale models such as large language models (LLMs).

Existing software stacks for IMC-based accelerators

Table 1 provides an overview and comparison of different compilation tools that can be used to automatically deploy general-purpose kernels and DL workloads to IMC-based accelerators. Numerous observations can be made. Many of these tools, despite being published, are closed source. Although most of these tools interface directly with popular machine-learning frameworks, the interoperability and infrastructure reuse could be greatly improved. Some existing infrastructure is reused and repurposed by many of these tools, but is done inconsistently. This fragmentation hinders interoperability and forces developers to tailor their workflows to a specific infrastructure. Hence, there are limited instances of these tools being publicly built upon and improved by other groups.

Benchmarking deep learning software stacks for AIMC-based accelerators

Benchmarking AIMC-based architectures is challenging owing to their diversity, limited hardware availability and closed source ecosystems.

Many of the existing DL software stacks for AIMC-based accelerators perform a combination of both quantitative and qualitative comparisons, which are not conducted under standardized or impartial conditions. Quantitative comparisons are made using the end-to-end performance of the configured accelerator, such as the tokens per second or tera operations per second per watt, but they are often normalized to a single point of reference. Qualitative comparisons typically assess methodologies for specific compilation steps, such as mapping and placement, but these evaluations are often superficial. Beyond ensuring fairer comparisons, an important yet frequently overlooked factor is the compiler's performance, including the gap between the peak and sustained tera operations per second per watt of the configured accelerator and the time required for compilation.

Research opportunities

AIMC-based accelerators are actively being developed, and building robust DL software stacks for these accelerators is crucial to fully realize their potential. A robust and engaged community is also needed, for innovation and long-term sustainability, and can be encouraged through open source initiatives, educational resources and community engagement.

Topics associated with the design and development of DL software stacks for AIMC-based accelerators are promising and highly impactful areas for research. To this end, numerous future research directions can be pursued, which we group into the following five broad themes.

Hardware integration and ecosystem development

The adoption of AIMC-based hardware accelerators for DL depends on their seamless integration into existing frameworks such as PyTorch and TensorFlow. Well-documented back-end integrations enable transparent execution and simplify compiler tool development. Similar to how Google's XLA is optimized for tensor processing units and NVIDIA's TensorRT for GPUs, achieving deep integration would allow machine-learning engineers to experiment with AIMC-based accelerators, fostering innovation and improving system-level performance and model accuracy.

Intermediate representations, optimization and compiler tools

Existing IRs for DNN models with weights mapped to AIMC tiles lack widespread adoption and consistent use across different compilation tools. This is due to use of closed source tools, hardware-specific proprietary representations and the need for multiple dialects at different compilation stages. A major challenge is to balance abstraction and hardware specificity. High-level IRs enhance portability but lack fine control, whereas low-level IRs are optimized but less generalizable. A hierarchical IR structure with a well-defined lowering process is ideal. Open source efforts are crucial to standardizing IRs for AIMC, with frameworks such as MLIR providing a strong foundation. However, further work is needed to develop pattern-matching libraries and portable optimizations, fostering a unified ecosystem for AIMC-based accelerators.

For AIMC-based accelerators, during compilation, steps that govern placement, routing or scheduling – for example, the mapping of DNNs to processing units – are highly interdependent. As a result, decisions made in one step influence the constraints and the achievable performance and accuracy in others. For instance, optimal mapping may increase communication demands, requiring careful placement and routing; poor placement or excessive data traffic can cause scheduling conflicts. Achieving a near-optimal solution often requires iterative

refinement or the use of joint optimization strategies to balance these interdependencies.

Another research topic is inter-layer pipelining. Most DL software stacks are designed to perform intra-layer parallelism, in which one layer is executed at a time, and the same processing units are reused for different layers. Adapting these stacks to support the pipelining of different input batches across layers is challenging, as it deviates from the standard execution model. Although some compilation tools, such as the Pipeline Executor in Relay, have introduced experimental support for this type of pipelining to accommodate emerging custom hardware accelerators, the functionality remains in its early stages.

Placement, scheduling and routing

During compilation, the implications of data movement between AIMC tiles and DPUs must be taken into consideration – particularly with regard to the impact on placement, scheduling and routing. As the resulting design space is extremely large, most previous work^{9,35–37} has focused on one of these three aspects, while making simplifying assumptions about the others. Taking into account the interdependency of placement, routing and scheduling is non-trivial, and although some works have explored this using accurate simulation-based tools^{38–40} and analytical models^{35,36}, integrating such approaches into a DL software stack for AIMC-based hardware and evaluating their effectiveness remains an open area of research.

Additionally, to achieve optimal performance when weights are stationary, pipelined execution across layers as well as within a layer is required – that is, intra-module and inter-module parallelism. In a fully weight stationary accelerator, input tensors should enter the system such that when the first vector finishes with the first layer (or the first part of the first layer), another can immediately take its place while the first vector continues its journey through the spatially mapped network. To achieve such a stall-free pipeline, balancing the number of operations across the layers may also be necessary – for example, by replicating the weights of a small but heavily used layer, such as small convolutional kernels.

HWA model adaptations and refinement of chip configuration

To achieve satisfactory DNN accuracy, a combination of HWA training, post-placement calibration methods and chip-in-the-loop retraining is crucial for AIMC-based accelerators. Libraries for HWA retraining and other HWA model adaptations on AIMC-based architectures exist but require extensive hyperparameter tuning and manual intervention, making automation difficult. This complexity arises from the large number of additional hyperparameters and constraints introduced during retraining, which increase sensitivity to scale and weight perturbations. Unlike quantization-aware training, HWA retraining provides robustness against non-idealities that are non-deterministic and potentially evolving over time. It is also computationally expensive, especially for large models or when heterogeneous mapping is involved. Although some progress has been made to simplify the process and increase training speed⁴¹, HWA retraining remains challenging, with major work needed to make it more efficient and automated. Post-placement calibration methods⁴² help mitigate accuracy loss caused by mismatches between the assumed hardware during HWA model adaptations and the actual hardware used for deployment. Their main advantage is avoiding the need for costly iterative HWA retraining during the compilation process.

In contrast, chip-in-the-loop retraining^{43,44} directly compensates for device non-idealities by performing on-chip MVMs during forward propagations and updating the devices using the accumulated

gradients of the DNN loss function through means of back propagation. The primary benefits of such a strategy are that it can easily be automated for different hardware architectures and does not rely on hardware-specific information. However, chip-in-the-loop retraining often requires more complex circuitry and additional memory, compared with standard programming circuits, to continuously update weights, gradients and other parameters. Moreover, chip-in-the-loop retraining is prone to overfitting, is hardware instance-specific, can be limited by the fixed endurance of the AIMC devices and can be difficult to scale for especially large networks, for example, LLMs. Further research into improving stability and optimizing computational efficiency will be crucial for making chip-in-the-loop training a viable solution in real-time deployment scenarios for AIMC accelerators.

Model serving

Finally, developing a DL software stack requires careful consideration of the programming mode and model-serving scenarios. Offline, single-user model serving is straightforward, but serving multiple users, handling asynchronous requests or distributing execution across accelerators adds complexity. Existing frameworks can manage distributed execution and online model serving but often require access to the hardware's internal state, making integration challenging. Serving LLMs poses unique challenges owing to their large size and the need for efficient key-value cache management to support low-latency, high-throughput inference. Frameworks such as vLLM and TensorRT-LLM optimize memory and inference but require further improvements to fully support heterogeneous accelerators and enhance the workload distribution across accelerators.

Published online: 2 July 2025

References

- Wu, C.-J. et al. Sustainable AI: environmental implications, challenges and opportunities. In *Proc. Machine Learning and Systems 4* (eds Marculescu, D. et al.) 795–813 (2022).
- Chen, Y., Xie, Y., Song, L., Chen, F. & Tang, T. A survey of accelerator architectures for deep neural networks. *Engineering* **6**, 264–274 (2020).
- Sebastian, A., Gallo, M. L., Khaddam-Aljameh, R. & Eleftheriou, E. Memory devices and applications for in-memory computing. *Nat. Nanotechnol.* **15**, 529–544 (2020). <https://doi.org/10.1145/3079856.3080246> (IEEE, 2017).
- Jouppi, N. P. et al. In-datacenter performance analysis of a tensor processing unit. In *2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA)* <https://doi.org/10.1145/3079856.3080246> (IEEE, 2017).
- Ambrogio, S. et al. An analog-AI chip for energy-efficient speech recognition and transcription. *Nature* **620**, 768–775 (2023).
- Guo, A. et al. 4.3 A 22nm 64kb Lightning-like hybrid computing-in-memory macro with a compressed adder tree and analog-storage quantizers for transformer and CNNs. In *IEEE International Solid-State Circuits Conference (ISSCC)* Vol. 67 570–572 (IEEE, 2024).
- Aguirre, F. et al. Hardware implementation of memristor-based artificial neural networks. *Nat. Commun.* **15**, 1974 (2024).
- Huang, Y. et al. Memristor-based hardware accelerators for artificial intelligence. *Nat. Rev. Electr. Eng.* **1**, 286–299 (2024).
This review covers the latest progress in memristive crossbar arrays, peripheral circuits, architectures, hardware–software co-designs and system implementations for memristor-based hardware accelerators.
- Jain, S. et al. A heterogeneous and programmable compute-in-memory accelerator architecture for analog-AI using dense 2-D mesh. *IEEE Trans. Very Large Scale Integr. VLSI Syst.* **31**, 114–127 (2023).
- Burr, G. W. et al. Design of analog-AI hardware accelerators for transformer-based language models. In *2023 International Electron Devices Meeting (IEDM)* <https://doi.org/10.1109/IEDM45741.2023.10413767> (2023).
- Krestinskaya, O. et al. Neural architecture search for in-memory computing-based deep learning accelerators. *Nat. Rev. Electr. Eng.* **1**, 374–390 (2024).
This review presents applications of hardware neural architecture search to the specific features of IMC hardware and compares existing optimization frameworks.
- Gallo, M. L. et al. A 64-core mixed-signal in-memory compute chip based on phase-change memory for deep neural network inference. *Nat. Electron.* **6**, 680–693 (2023).
- Boybat, I. et al. Heterogeneous embedded neural processing units utilizing PCM-based analog in-memory computing. In *2024 IEEE International Electron Devices Meeting (IEDM)* <https://doi.org/10.1109/IEDM50854.2024.10873479> (IEEE, 2024).
- Paszke, A. et al. Automatic differentiation in PyTorch. In *NIPS 2017 Workshop on Autodiff* (2017).
- Li, M. et al. The deep learning compiler: a comprehensive survey. *IEEE Trans. Parallel Distrib. Syst.* **32**, 708–727 (2021).
This paper presents a comprehensive survey of deep learning compilation.
- Lattner, C. & Pienaar, J. MLIR primer: a compiler infrastructure for the end of Moore's Law. Preprint at <https://arxiv.org/abs/2002.11054> (2019).
This paper introduces the MLIR project, which is a novel approach to building reusable and extensible compiler infrastructure.
- Pichler, C., Li, P., Schatz, R. & Mössenböck, H. Hybrid execution: combining ahead-of-time and just-in-time compilation. In *Proc. 15th ACM SIGPLAN International Workshop on Virtual Machines and Intermediate Languages* 39–49 (Association for Computing Machinery, 2023).
- He, K., Chakraborty, I., Wang, C. & Roy, K. Design space and memory technology co-exploration for in-memory computing based machine learning accelerators. In *ICCAD '22: Proc. 41st IEEE/ACM International Conference on Computer-Aided Design* <https://doi.org/10.1145/3508352.3549453> (Association for Computing Machinery, 2022).
- Sharma, S. & Kumar, V. A comprehensive review on multi-objective optimization techniques: past, present and future. *Arch. Comput. Methods Eng.* **29**, 5605–5633 (2022).
- Abadi, M. TensorFlow: learning functions at scale. In *ICFP 2016: Proc. 21st ACM SIGPLAN International Conference on Functional Programming* <https://doi.org/10.1145/2951913.2976746> (Association for Computing Machinery, 2016).
- Roesch, J. et al. Relay: a new IR for machine learning frameworks. In *MAPL 2018: Proceedings of the 2nd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages* 58–68 (Association for Computing Machinery, 2018).
- Cyphers, S. et al. Intel nGraph: an intermediate representation, compiler, and executor for deep learning. Preprint at <http://arxiv.org/abs/1801.08058> (2018).
- Chen, T. et al. TVM: an automated end-to-end optimizing compiler for deep learning. In *OSDI'18: Proc. 13th USENIX conference on Operating Systems Design and Implementation* 579–594 (USENIX Association, 2018).
- Lattner, C. & Adiv, V. LLVM: a compilation framework for lifelong program analysis & transformation. In *International Symposium on Code Generation and Optimization*, 2004 75–86 (IEEE, 2004).
- Rotem, N. et al. Glow: Graph lowering compiler techniques for neural networks. Preprint at <http://arxiv.org/abs/1805.00907> (2018).
- Jeong, E., Kim, J. & Ha, S. TensorRT-based framework and optimization methodology for deep learning inference on Jetson boards. *ACM Trans. Embedded Computer Systems* <https://doi.org/10.1145/3508391> (2022).
- Demidovskij, A. et al. OpenVINO Deep Learning Workbench: a platform for model optimization, analysis and deployment. In *2020 IEEE 32nd International Conference on Tools with Artificial Intelligence (ICTAI)* 661–668 (IEEE, 2020).
- Lammie, C. et al. LIONHEART: a layer-based mapping framework for heterogeneous systems with analog in-memory computing tiles. *IEEE Transactions on Emerging Topics in Computing* <https://ieeexplore.ieee.org/document/10910024> (IEEE, 2025).
- Khaddam-Aljameh, R. et al. HERMES core – a 14 nm CMOS and PCM-based in-memory compute core using an array of 300ps/LSB linearized CCO-based ADCs and local digital processing. In *2021 Symposium on VLSI Technology* <https://ieeexplore.ieee.org/document/9508706> (IEEE, 2021).
- Lammie, C., Büchel, J., Vasilopoulos, A., Le Gallo, M. & Sebastian, A. The inherent adversarial robustness of analog in-memory computing. *Nat. Commun.* **16**, 1756 (2025).
- Rasch, M. J. et al. Hardware-aware training for large-scale and diverse deep learning inference workloads using in-memory computing-based accelerators. *Nat. Commun.* **14**, 5282 (2023).
- Si, M., Cheng, H.-Y., Ando, T., Hu, G. & Ye, P. D. Overview and outlook of emerging non-volatile memories. *MRS Bull.* **46**, 946–958 (2021).
- Ferro, E. et al. A precision-optimized fixed-point near-memory digital processing unit for analog in-memory computing. In *IEEE International Symposium on Circuits and Systems (ISCAS)* <https://ieeexplore.ieee.org/document/10558286> (IEEE, 2024).
- Benmeziane, H. et al. Multi-task neural network mapping onto analog-digital heterogeneous accelerators. In *38th Second Workshop on Machine Learning with New Compute Paradigms* (2024).
- Andrulis, T., Emer, J. S. & Sze, V. CiMLoop: a flexible, accurate, and fast compute-in-memory modeling tool. In *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)* 10–23 (IEEE, 2024).
- Roux, B., Gautier, M., Sentieys, O. & Delahaye, J.-P. Energy-driven design space exploration of tiling-based accelerators for heterogeneous multiprocessor architectures. *MICPRO or Microprocess.* **77**, 103138 (2020).
- Wang, L., Tarnawski, J., Phanishayee, A. & Mahajan, D. Integrated hardware architecture and device placement search. In *ICML'24: Proc. 41st International Conference on Machine Learning* (eds Salakhutdinov, R. et al.) 51523–51545 (JMLR.org, 2024).
- Cong, J., Fang, Z., Gill, M. & Reinman, G. PARADE: a cycle-accurate full-system simulation platform for accelerator-rich architectural design and exploration. In *2015 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)* 380–387 (IEEE, 2015).
- Shao, Y. S., Xi, S. L., Srinivasan, V., Wei, G.-Y. & Brooks, D. Co-designing accelerators and SoC interfaces using gem5-Aladdin. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)* <https://ieeexplore.ieee.org/document/7783751> (IEEE, 2016).
- Klein, J. et al. ALPINE: analog in-memory acceleration with tight processor integration for deep learning. *IEEE Trans. Comput.* **72**, 1985–1998 (2023).

41. Büchel, J. et al. AIHWKIT-Lightning: a scalable HW-aware training toolkit for analog in-memory computing. In *NeurIPS Workshop Machine Learning with new Compute Paradigms* <https://neurips.cc/virtual/2024/101271> (NeurIPS, 2024).
42. Lammie, C. et al. Improving the accuracy of analog-based in-memory computing accelerators post-training. In *2024 IEEE International Symposium on Circuits and Systems (ISCAS)* <https://ieeexplore.ieee.org/document/10558540> (IEEE, 2024).
43. Yao, P. et al. Fully hardware-implemented memristor convolutional neural network. *Nature* **577**, 641–646 (2020).
44. Wan, W. et al. A compute-in-memory chip based on resistive random-access memory. *Nature* **608**, 504–512 (2022).
45. Yu, J., Hogervorst, T. & Nane, R. A domain-specific language and compiler for computation-in-memory skeletons. In *GLSVLSI '17: Proc. Great Lakes Symposium on VLSI 2017* 71–76 (Association for Computing Machinery, 2017).
This paper introduces the first compiler for IMC hardware.
46. Chakraborty, D., Raj, S., Gutierrez, J. C., Thomas, T. & Jha, S. K. In-memory execution of compute kernels using flow-based memristive crossbar computing. In *2017 IEEE International Conference on Rebooting Computing (ICRC)* <https://ieeexplore.ieee.org/document/8123643> (IEEE, 2017).
47. Fujiki, D., Mahlke, S. & Das, R. In-memory data parallel processor. In *ASPLOS '18: Proc. Twenty-Third International Conference on Architectural Support for Programming Languages and Operating Systems* <https://doi.org/10.1145/3173162.3173171> (Association for Computing Machinery, 2018).
48. Vadivel, K. et al. TDO-CIM: transparent detection and offloading for computation in-memory. In *2020 Design, Automation Test in Europe Conference & Exhibition (DATE)* 1602–1605 (IEEE, 2020).
49. Siemieniuk, A. et al. OCC: an automated end-to-end machine learning optimizing compiler for computing-in-memory. *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* **41**, 1674–1686 (2022).
50. Ambrosi, J. et al. Hardware–software co-design for an analog-digital accelerator for machine learning. In *2018 IEEE International Conference on Rebooting Computing (ICRC)* <https://ieeexplore.ieee.org/document/8638612> (2018).
51. Peng, X., Huang, S., Jiang, H., Lu, A. & Yu, S. DNN+NeuroSim V2.0: an end-to-end benchmarking framework for compute-in-memory accelerators for on-chip training. *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* **40**, 2306–2319 (2020).
52. Ankit, A. et al. PUMA: A programmable ultra-efficient memristor-based accelerator for machine learning inference. In *Proc. 24th International Conference on Architectural Support for Programming Languages and Operating Systems* 715–731 (Association for Computing Machinery, 2019).
53. Drebes, A. et al. TC-CIM: empowering tensor comprehensions for computing-in-memory. In *IMPACT 2020 10th International Workshop on Polyhedral Compilation Techniques* (2020).
54. Park, J. & Sung, H. XLA-NDP: efficient scheduling and code generation for deep learning model training on near-data processing memory. *IEEE Computer Architecture Letters* **22**, 61–64 (2023).
55. Jin, H. et al. A compilation tool for computation offloading in ReRAM-based CIM architectures. *ACM Trans. Architecture and Code Optimization* **20**, 1–25 (2023).
56. Delm, J. V. et al. HTVM: efficient neural network deployment on heterogeneous TinyML platforms. In *2023 60th ACM/IEEE Design Automation Conference (DAC)* <https://ieeexplore.ieee.org/document/10247664> (IEEE, 2023).
57. Sun, X., Wang, X., Li, W., Han, Y. & Chen, X. PIMCOMP: an end-to-end DNN compiler for processing-in-memory accelerators. *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* **44**, 1745–1759 (2024).
58. Bai, Y. et al. A compilation framework for SRAM computing-in-memory systems with optimized weight mapping and error correction. *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* **43**, 2379–2392 (2024).
59. Qu, S. et al. CIM-MLC: a multi-level compilation stack for computing-in-memory accelerators. In *Proc. 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* Vol. 2 185–200 (ACM, 2024).

Acknowledgements

This work was supported by the IBM Research AI Hardware Center.

Author contributions

C.L. conceptualized the article. C.L., with the help of H.B. and A.S., drafted the manuscript. W.S., E.F., A.V., J.B., M.L.G. and I.B. contributed to discussions and assisted in editing the manuscript. All authors reviewed and approved the final version of the article.

Competing interests

The authors declare no competing interests.

Additional information

Peer review information *Nature Reviews Electrical Engineering* thanks Huaqiang Wu and Jianhua Yang for their contribution to the peer review of this work.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

© Springer Nature Limited 2025